

Media Ratings Platform

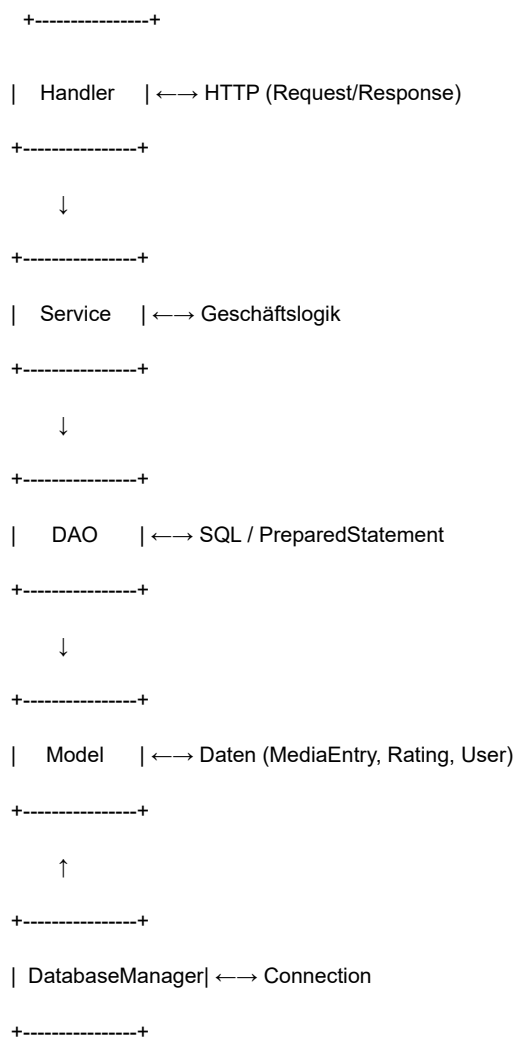
Schichtenarchitektur (Handler → Service → DAO → Model)

Handler: nur Routing & HTTP → wandelt Request in Objekte um, ruft Service auf, baut Response.

Service: reine Geschäftslogik — z. B. „Darf User A das Rating von User B löschen?“ → keine DB-Zugriffe.

DAO: nur SQL + PreparedStatement — jede Tabelle hat ihre eigene DAO Klasse.

Model: reine Datenklassen (POJOs), **keine Logik**, keine DB-Abhängigkeiten.



Besonderheiten: Kein direkter DB-Zugriff in Handlers / Keine SQL in Services / Keine HTTP-Details in Models

Merkmale: Extrem Hilfreich beim Debuggen. Als Ratings nicht gespeichert wurden, wusste ich sofort, nicht im Handler ist es kaputt, nicht im Service, sondern im DAO prüfen.

Lessons Learned:

Fehler 1: „mediald is null“ bei Rating-Erstellung

```
{"error": "Request failed: Cannot invoke \"java.lang.Integer.intValue()\" because the return value of \"model.Rating.getMediald()\" is null"}
```

→ Weil ich /api/ratings genutzt habe, aber mein RatingHandler nur /api/media/123/ratings akzeptiert hat.

Lösung:

Zwei Endpoints implementiert:

/api/media/123/ratings → mediald aus Pfad (/media/**123**/ratings)

/api/ratings → mediald aus JSON-Body ({"mediald":123,...})

Beides über eine gemeinsame Hilfsmethode abgewickelt

Fehler 2: DB Cleanup schlägt fehl bei DROP TABLE ratings

Meldung:

ERROR: cannot drop table ratings because other objects depend on it

→ Weil ich später die likes-Tabelle hinzugefügt hatte (für das Like-Feature), aber im cleanDB()-Test nicht mit CASCADE gelöscht.

Lösung:

In AllTests.java alle DROP TABLE mit CASCADE versehen — und Reihenfolge umgedreht (zuerst likes, dann favorites, dann ratings, ...)

Unit-Testing-Strategie

Ich hab die 20 Unittesten in 4 Gruppen unterteilt: User, Media, Rating und Profil, jeweils mit 5 Tests.

Testgruppe	Anzahl	Was getestet?	Beispiel
User	5	Register, Login, Duplikate, Token-Validierung	testUserRegisterDuplicate() → wirft Exception bei doppeltem User
Media	5	CRUD, Filter nach Titel	testMediaList() → prüft, ob "A Movie" bei Filter "A" gefunden wird
Rating	5	Erstellen, Duplikat, Moderation, Bestätigung, Durchschnitt	testConfirmRatingByCreator() → erst unbestätigt → dann bestätigt → dann sichtbar
Favoriten & Profil	5	Favoriten hinzufügen/entfernen, Profil-Statistiken	testFavoriteGenreCalculation() → prüft, ob nach 2 Sci-Fi-Bewertungen "sci-fi" als Lieblingsgenre kommt

SOLID Prinzipien Anwendung:

Jede Klasse hat **genau eine Aufgabe** und zwar so getrennt, dass ich sie wie folgt beschreiben kann:

Klasse	Verantwortung
MediaHandler	HTTP-Requests für Medien entgegennehmen & weiterleiten
MediaService	„Darf User X Medium Y löschen?“, reine Logik, keine DB
MediaDao	Nur: SQL ausführen, ResultSet → MediaEntry mappen
RatingService	Nur: „Ist das Rating editierbar?“, „Darf bestätigt werden?“

In MediaHandler.handleUpdate() prüfe ich nicht selbst, ob der User der Ersteller ist, das macht MediaService.update(). Der Handler ruft nur service.update(...) auf und fängt ggf. eine IllegalArgumentException ab, dadurch saubere Trennung.

Man sieht dass zum Thema jeweiligen für jeden Verwendungszweck und Logik ganz strikt andere Klassen zum gelten kommen.

Zeittracking:

Aufgabe	Geschätzte Zeit	Tatsächliche Zeit
DB-Setup (Schema, Tabellen, Constraints)	2 h	4 h
HTTP-Server & Request/Response Mapping	3 h	5 h
User-Registrierung & Login (Token Verwaltung)	2 h	5 h
Media CRUD	3 h	4 h
Search & Filter	2 h	3 h
Like Funktion & Edit/Delete Rating	3 h	4 h
Unit Tests schreiben & debuggen	4 h	8 h
Integrationstests (test.bat)	1,5 h	4 h